Point elevations from survey differences

Gabriel Arellano

2026-11-11

Contents

1 Purpose **1**

2 Read the survey and estimate elevations **2**
2.1 Survey lines 3
2.2 Estimate elevations 4

3 Compare with Condit's solver **5**

4 Small checks for the old problems **7**

1 Purpose

This solver estimates one elevation per point from measured height differences between pairs of points. It follows Condit's equal-weight least-squares calculation.

Height differences can come from establishing a plot grid. A team marks points with stakes and measures the vertical difference between pairs, with different instruments. The coordinates may also describe other surveyed locations, not necessarily stakes in a regular grid. The solver works on irregularly spaced points and does not require a complete regular grid. See Condit's survey example for an overview of the process, using the old tools and conventions.

This is a limited tool for pairwise survey records, with little consideration for error propagation and so forth. For a new survey, professional teams or drone mapping may provide a more detailed elevation model.

2 Read the survey and estimate elevations

The supplied CSV is simulated on a 500 by 500 m plot (25 ha). The principal 20 m grid has 676 planned points. Seven of these (about 1%) are unusable; short routes with 5 m and 10 m measurements pass around them. A locally rugged area has additional 10 m measurements and a smaller patch with 5 m infill. The file contains 878 measured points and 1,737 observations, including repeats, small disagreements and mixed measurement directions. All coordinates and height differences are in metres.

The default paths below work when rendering this tutorial or running from the repository root. Set a full path to fgeo2 when using the code elsewhere.

```
# install.packages(c("Matrix", "igraph", "jsonlite"))
path.to.fgeo2 = if(file.exists("../../fgeo2.Rproj")) "../.." else "."
# path.to.fgeo2 = "/full/path/to/fgeo2"

records <- read.csv(file.path(path.to.fgeo2, "data_examples/topography-survey.csv"))

# One row per measured difference. Positive dz means the destination is higher.
survey <- data.frame(
    x_i = records$from_x_m,
    y_i = records$from_y_m,
    x_j = records$to_x_m,
    y_j = records$to_y_m,
    dz = records$height_difference_m
)
head(survey)
```

```
  x_i y_i x_j y_j      dz
1 255 240 250 240 -1.9133
2   0 400  20 400 -0.0908
3 240 200 230 200 -0.3769
4 275 275 280 275 -0.5853
5 240 200 240 180 -0.0555
6 180 140 180 160 -0.4326
```

```
# Horizontal distances between the two surveyed locations.
survey.distance <- sqrt((survey$x_j - survey$x_i)^2 + (survey$y_j - survey$y_i)^2)
table(survey.distance)
```

```
survey.distance
```

```
  5   10   20
191  264 1282
```

2.1 Survey lines

Each line joins a pair of surveyed points, as in section 2 of Condit's tutorial. The full plot shows the 20 m grid; the detail shows short routes and the denser survey in the rugged area. Red crosses mark the seven planned points that could not be used.

```
measured <- unique(rbind(
    data.frame(x = survey$x_i, y = survey$y_i),
    data.frame(x = survey$x_j, y = survey$y_j)
))
grid <- expand.grid(x = seq(0, 500, 20), y = seq(0, 500, 20))
unusable <- !paste(grid$x, grid$y) %in% paste(measured$x, measured$y)

settings <- par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
views <- list("Whole plot" = c(0, 500), "Local detail" = c(200, 300))
for(view in names(views))
{
    plot(measured$x, measured$y, type = "n", asp = 1,
         xlim = views[[view]], ylim = views[[view]],
         xlab = "x (m)", ylab = "y (m)", main = view)
    segments(survey$x_i, survey$y_i, survey$x_j, survey$y_j,
             col = ifelse(survey.distance < 20, "steelblue", "white"))
    points(measured$x, measured$y, pch = 16, cex = 1)
    points(grid$x[unusable], grid$y[unusable], pch = 4, col = "firebrick", lwd = 1.5)
}
par(settings)
```

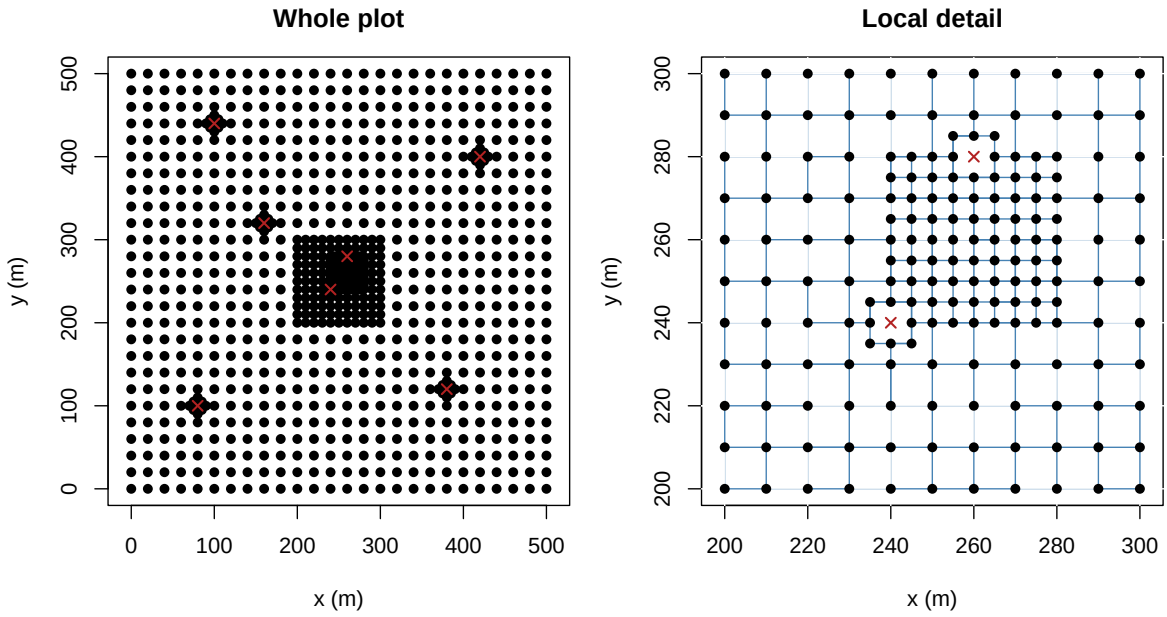


Figure 1: Simulated 25 ha survey. Blue lines show 5 m or 10 m measurements; grey lines show 20 m measurements. Red crosses indicate unusable grid points.

2.2 Estimate elevations

Set the coordinates and elevation of the reference point, then fit all measured differences together.

```
# Assign this elevation to the point at these coordinates.
reference.xy = c(0, 0)
reference.z = 100  # Use 0 for relative elevations if absolute elevation is unknown.

source(file.path(path.to.fgeo2, "06_topography_habitat/R/new/topography-solver.R"))
elevations <- estimate_point_elevations(survey, reference.xy, reference.z)
head(elevations)
```

```
  x   y        z
1 0   0 100.0000
2 0  20 100.2636
3 0  40 100.5678
4 0  60 100.8437
5 0  80 101.1900
6 0 100 101.5128
```

Other file layouts must be processed into the same five-column `survey` table. The solver identifies points by exact coordinate pairs, so use consistent coordinates for repeated points. The point IDs in this example file are only used by the old solver below.

Rows and columns can be in any order. Repeated and conflicting measurements receive equal weight. Incomplete rows are omitted with a warning. Missing measurements or missing grid points are acceptable while the remaining observations connect every retained point. Points represented only by omitted rows disappear from the output. The result contains `x`, `y` and `z`; raster construction and interpolation are separate steps.

3 Compare with Condit's solver

The old code is read from fgeo2 as it stood through 2026-11-09 (UTC), before this replacement. This avoids depending on copies in the current `R/old` or `evaluation/Condit` folders. The date lookup needs internet access and `jsonlite`.

```
source(file.path(path.to.fgeo2, "evaluation/0-code-to-source.r"))
old <- new.env()
load_github_snapshot(
    commit.date = "2026-11-09",
    scripts = "06_topography_habitat/R/new/topography-solvetopo.r",
    envir = old
)
```

```
gabrielareto/fgeo2 commit: 0a2c01dd560daacba0fdc2eb8c9280e2b9d2ea9d
```

```
# If needed, replace the call above with ONE alternative below.
# source(file.path(path.to.fgeo2, "06_topography_habitat/R/old/topography-solvetopo.r"), l
# source(file.path(path.to.fgeo2, "evaluation/Condit/topography/solvetopo.r"), local = old
# old$solve.topo <- ctfs::solve_topo  # Requires the ctfs package.
```

First compare relative elevations at datum zero. Both solvers should reconcile the measurements in the same way.

```
# Condit's solver needs point IDs and a coordinate lookup.
points <- unique(rbind(
    data.frame(id = records$from_point, x = records$from_x_m, y = records$from_y_m),
    data.frame(id = records$to_point, x = records$to_x_m, y = records$to_y_m)
))
points <- points[order(points$x, points$y), ]
```

```
  base.id <- points$id[points$x == reference.xy[1] & points$y == reference.xy[2]]

  old.survey <- data.frame(
      pt1 = records$from_point,
      pt2 = records$to_point,
      htdiff = records$height_difference_m
  )
  old.fit <- old$solve.topo(old.survey, basept = base.id, baseelev = 0)
```

```
at point  1
at point  101
at point  201
at point  301
at point  401
at point  501
at point  601
at point  701
at point  801
```

```
  old.z <- old.fit$elev[match(points$id, old.fit$label)]
  new.fit <- estimate_point_elevations(survey, reference.xy, reference.z = 0)

  # Align by coordinates, then check agreement.
  stopifnot(all(points$x == new.fit$x), all(points$y == new.fit$y))
  max(abs(old.z - new.fit$z))
```

```
[1] 3.872458e-13
```

```
  stopifnot(isTRUE(all.equal(old.z, new.fit$z, tolerance = 1e-8)))
```

The two panels show the fitted terrain at the retained 20 m grid points. The seven unusable corners are left as gaps. All measurements, including the short-distance observations, contributed to the fit. Both panels use the same scales and viewing angles. R perspective plots

```
  x <- seq(0, 500, 20)
  y <- seq(0, 500, 20)
  old.surface <- new.surface <- matrix(NA_real_, length(x), length(y))
```

```
# Show the nominal 20 m grid, leaving unsurveyed corners as NA.
# Matrix rows correspond to x and columns to y.
keep.old <- points$x %in% x & points$y %in% y
keep.new <- new.fit$x %in% x & new.fit$y %in% y
old.surface[cbind(match(points$x[keep.old], x), match(points$y[keep.old], y))] <- old.z[ke
new.surface[cbind(match(new.fit$x[keep.new], x), match(new.fit$y[keep.new], y))] <- new.fi
z.limits <- range(old.z, new.fit$z)

settings <- par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
persp(x, y, old.surface, zlim = z.limits, theta = 35, phi = 25,
      scale = FALSE, col = "lightblue", shade = 0.4,
      xlab = "x (m)", ylab = "y (m)", zlab = "z (m)", main = "Condit")
persp(x, y, new.surface, zlim = z.limits, theta = 35, phi = 25,
      scale = FALSE, col = "lightblue", shade = 0.4,
      xlab = "x (m)", ylab = "y (m)", zlab = "z (m)", main = "New solver")
par(settings)
```

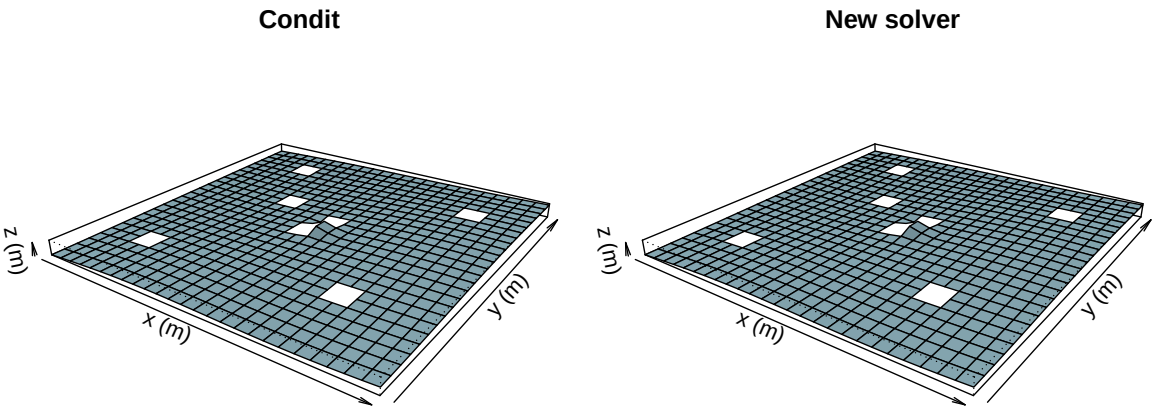


Figure 2: Relative elevations from the same survey. Condit's solver and the new solver agree.

4 Small checks for the old problems

A chain with differences of 2 and 3 m should give elevations 100, 102 and 105 m when its first point is at 100 m. The old code applies the datum only to the first point. It also mishandles reordered columns and propagates a missing difference. These small examples show the fixes.

```r
chain <- data.frame(x_i = c(0, 1), y_i = 0, x_j = c(1, 2), y_j = 0, dz = c(2, 3))
old.chain <- data.frame(pt1 = c("1", "2"), pt2 = c("2", "3"), htdiff = c(2, 3))

# A nonzero datum must shift every point.
rbind(
    Condit = old$solve.topo(old.chain, baseelev = 100)$elev,
    New = estimate_point_elevations(chain, reference.z = 100)$z
)
```

```
at point  1
```

```
       [,1] [,2] [,3]
Condit  100    2    5
New     100  102  105
```

```r
# Reordering columns must preserve the direction of the measured differences.
rbind(
    Condit = old$solve.topo(old.chain[c("pt2", "pt1", "htdiff")])$elev,
    New = estimate_point_elevations(chain[c("dz", "x_j", "y_j", "x_i", "y_i")])$z
)
```

```
at point  1
```

```
       [,1] [,2] [,3]
Condit    0   -2   -5
New       0    2    5
```

```r
# An extra row with a missing difference must not contaminate usable measurements.
rbind(
    Condit = old$solve.topo(rbind(old.chain, transform(old.chain[1, ], htdiff = NA)))$elev
    New = estimate_point_elevations(rbind(chain, transform(chain[1, ], dz = NA)))$z
)
```

```
at point  1

Warning: 1 incomplete or same-point observation(s) omitted.
```

```
       [,1] [,2] [,3]
Condit    0   NA   NA
New       0    2    5
```

```r
stopifnot(
    isTRUE(all.equal(estimate_point_elevations(chain, reference.z = 100)$z, c(100, 102, 10
    isTRUE(all.equal(estimate_point_elevations(chain[2:1, 5:1])$z, c(0, 2, 5)))
)
```